COSC 461/561 Project 1: Eagle Delivery

Arabi Rahman Rafi · https://github.com/arabirahmanrafi-svg/EagleDelivery

Part A: Planning a delivery with A*

Formulation. A *state* is the robot's cell (row, column) on any traversable square (walkway or building entrance). The *initial state* is the start entrance's cell, e.g. S = (1, 1). The *actions* are UP, DOWN, LEFT, RIGHT. The *transition model* is deterministic: a move shifts the robot one cell if the target is inside the map and not a wall (#), and otherwise leaves it in place. The *goal test* is position = destination entrance; other entrances are ordinary walkways. The *path cost* is the number of actions, each costing 1. Search only uses moves that change the position, because a blocked move costs 1 and returns to the same state, so it is never part of a cheapest route.

Why BFS is optimal. BFS removes states in order of depth, and it records each state the first time it reaches it, which is through a shallowest path. Every action costs 1, so depth equals path cost, and the first time the goal is removed it lies on a minimum-cost route. **Why Manhattan distance is admissible.** Each action changes the row or the column by at most 1, so any route from s to g needs at least $|r_s - r_g| + |c_s - c_g|$ actions. Walls only remove moves and there are no diagonals, so h(s) never overestimates the true remaining cost. It is also consistent: one unit-cost move changes h by at most 1.

Implementation. The heap holds (f, insertion number, g, state), so equal-f ties pop in insertion order. The `best_g` and `parents` dictionaries are updated whenever a strictly cheaper g is found, and a new entry is pushed. A popped entry whose g no longer matches `best_g` is stale and skipped. The goal test runs when an entry is removed, and a node counts as expanded when its successors are generated, the same convention as BFS.

Map	Delivery task	BFS length	A* length	BFS expanded	A* expanded
Baseline	S-W	18	18	67	22
Baseline	L-D	22	22	79	62
Baseline	H-P	16	16	41	16
Detour	S-W	26	26	70	51
Detour	L-D	22	22	65	60
Detour	H-P	16	16	41	16

Table 1. BFS vs A* on the six required tasks (results/search_results.txt).

Discussion. Both algorithms find routes of the same length in every case, as expected, since both are optimal here. A* expanded fewer nodes in all six cases, but how many fewer depends on how closely h matches the real distance. When the walls bend the route only a little, A* expands barely more than the route itself: baseline S-W has h(S) = 14 and cost 18 (22 vs 67 expansions), and H-P has h = 12 and cost 16 (16 vs 41). For L-D, h(L) = 14 but the cost is 22, because the route has to swing far away from the straight line. A* must then expand every state with f < 22, which covers much of the map (62 vs 79). The closure at (3, 6) in the detour map turns the corridor that h points toward into a dead end. On detour S-W, A* follows that corridor first and then backs out, so its expansions rise from 22 to 51 while BFS barely changes (67 to 70). Detour L-D is the closest case (60 vs 65). A* is not guaranteed to expand fewer nodes; a misleading heuristic and ties among states with f equal to the optimal cost can make it explore states BFS would not need.

Part B: Learning a delivery policy with Q-learning

Implementation. `epsilon_greedy` draws `rng.random()`; below ε it returns `rng.choice` over all four actions. Otherwise it finds the largest Q-value (unseen entries count as 0.0) and uses `rng.choice` to break ties among the actions that reach it. `q_learning_update` uses the target y = r for a terminal transition and $y = r + \gamma \max_{a'} Q(s', a')$ otherwise (unseen actions count as 0.0), then updates only $Q(s, a) \leftarrow Q(s, a) + \alpha(y - Q(s, a))$. All 14 public tests pass. Baseline settings: 2,000 training episodes, 100 greedy evaluation episodes, a 200-action cap, $\alpha = 0.3$, $\gamma = 0.95$, constant $\varepsilon = 0.2$, no slip, and seeds 0, 1, 2 with a fresh Q-table each.

Task	Optimal length	Eval success	Eval steps (return)	First success (ep.)	Success rate, eps 1-100	Truncated train eps	Train steps, last 500
S-W	18	100%	18 (22)	3.0 ± 2.6	0.83 ± 0.01	17.0 ± 1.0	22.5 ± 0.1
L-D	22	100%	22 (18)	3.7 ± 0.6	0.91 ± 0.01	8.7 ± 0.6	27.7 ± 0.1
H-P	16	100%	16 (24)	2.0 ± 0.0	0.85 ± 0.00	15.3 ± 0.6	20.1 ± 0.0

Table 2. Baseline Q-learning on emu_campus, mean ± sample SD over seeds 0, 1, 2 (from the training, evaluation, and summary CSVs).

Observations. All nine runs reach 100% evaluation success, and every evaluation episode follows a shortest route (18, 22, and 16 actions), so the returns equal 40 - L (22, 18, 24) with zero variance across seeds. Training goes through three phases. (1) *Exploration:* in the first 100 episodes many episodes hit the 200-action limit (8-18 truncated episodes per run, all within

the first 150), yet the first delivery comes within the first 6 episodes. The Q-values start at 0 and every step costs -1, so an action that has been tried quickly drops below 0 and the untried actions win the greedy choice; the agent sweeps the map systematically. (2) *Convergence:* the training curves (Figure 1) drop to a plateau by about episode 60 for H-P, 120-140 for S-W, and 140 for L-D, matching task difficulty: L-D has the longest route, and the +39 reward must propagate back through 22 states. After episode 150 every training episode succeeds for every task. (3) *Plateau:* training steps level off above the optimum (22.5 vs 18, 27.7 vs 22, 20.1 vs 16) because ε = 0.2 keeps adding random detours. Longer routes give exploration more chances to stray, so the gap is largest for L-D.

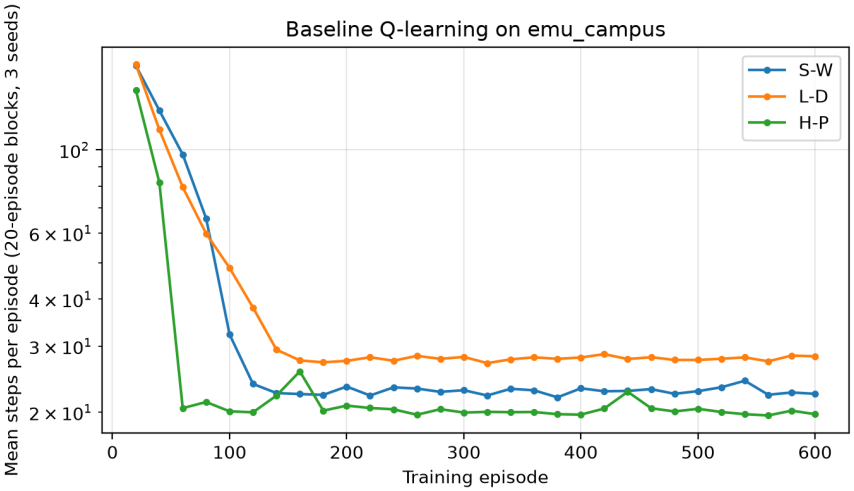


Figure 1. Mean training steps per episode in 20-episode blocks (averaged over 3 seeds, log scale) for the first 600 of 2,000 episodes.

Part C: Custom map and exploration experiment

```
####################
#S.....#.........#
#.#####.#.#########
#.#...#...#.....#.#
#.#.#.#####.###.#.#
#...#.....L.#H#...#
###.#######.#.#####
#.....#.....#.....#
#.###.#.#####.###.#
#.#D#...#...P...#.#
#.#.#####.#####.#W#
#.................#
####################
```

Map (maps/my_campus.txt, 13 × 19). S = (1, 1), W = (10, 17); 110 open cells and 112 connections, so 3 independent loops. **Dead ends:** the largest is the 20-cell branch that starts right next to S: it runs east along row 1, dips through (3, 7)-(3, 9), and continues east to (1, 17), right above W's column, where it stops. Smaller dead ends lead to (3, 17), to H, and to D. **Loops:** a middle loop through L, and two loops on the east side around P and around the (7, 13)-(7, 17) corridor next to W. **Route choices:** every route goes down column 1. The shortest then follows row 11 east and steps up into W (31 actions). Going up through (10, 9) past P costs 35, and taking the corridor above W costs 37. **Search:** BFS 31 actions / 103 expanded; A* 31 / 80. h(S) = 25: along that branch f = g + h stays between 25 and 27, below the true cost of 31, so A* explores the whole dead-end branch before taking the real route.

Question and prediction. How does the exploration rate ε affect learning speed, the cost of training, and the final policy on this map? *Prediction:* higher ε would find the goal faster, but it would give lower training returns, because random moves keep pulling the agent off the route.

Method. I varied only ε (0.05, 0.2, 0.5) on the S-W task. Everything else stayed at the Part B defaults: α = 0.3, γ = 0.95, 2,000 training episodes, a 200-action cap, no slip, and seeds 0, 1, 2 with a fresh Q-table for every (setting, seed) pair, giving 9 training runs. Each run was evaluated for 100 episodes with ε = 0 and frozen Q-values. I compared the settings using the evaluation summaries and the raw training CSVs.

ε	Eval success / steps	First success (ep.)	Success rate, eps 1-100	Truncated train eps	Total train actions	Train steps, last 500	Train return, last 500	Best train episode
0.05	100% / 31	27.0 ± 9.5	0.24 ± 0.02	93.7 ± 2.9	85109 ± 455	32.7 ± 0.1	7.3 ± 0.1	31, 31, 31
0.2	100% / 31	21.3 ± 3.2	0.28 ± 0.02	95.7 ± 3.2	96880 ± 237	39.0 ± 0.1	1.0 ± 0.1	31, 31, 31
0.5	100% / 31	32.3 ± 11.0	0.20 ± 0.03	114.0 ± 6.6	146647 ± 1413	65.1 ± 0.4	-25.1 ± 0.3	36, 38, 39

Table 3. my_campus S-W by exploration rate (mean ± SD over seeds 0-2). Optimal route = 31 actions (return 9). "Best train episode" lists each seed's shortest training episode.

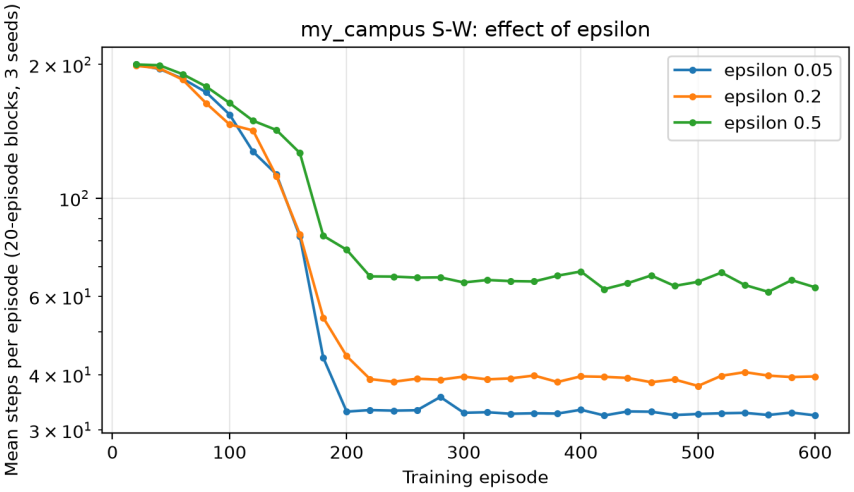


Figure 2. Mean training steps per episode (20-episode blocks, 3 seeds, log scale) for each ε.

Findings. (1) *Final policy: no difference.* All nine runs evaluate at 100% success along the 31-action route (return 9). (2) *Learning speed: little difference, so the first half of my prediction was wrong.* Higher ε did not find the goal sooner: the first delivery came at episode 27, 21, and 32 on average, and the seed-to-seed spread is larger than the gaps between settings. All three curves reach their plateau at about episode 180-220. (3) *Training cost: a large difference, as predicted.* As ε rises, total training actions grow from 85k to 97k to 147k, and late-training returns fall from +7.3 to +1.0 to -25.1. With ε = 0.5 the agent never walked the optimal route during training (its best episodes took 36-39 actions), and it still had occasional failed episodes late in training (as late as episode 1696). Each step picks the greedy action with probability 0.5 + 0.5/4 = 0.625, so a perfect 31-step episode has probability of only about $0.625^{31} \approx 5 \times 10^{-7}$. This map was also much harder than the campus map: in the first 100 episodes only 20-28% of episodes succeeded (vs 83-91% for the baseline tasks), because the 20-cell dead end starts right beside S and the route is 31 steps long.

Discussion. The surprise is that ε barely changed how fast the agent learned. Two properties of this setup explain it. First, with Q-values starting at 0 and a -1 reward per step, every tried action looks worse than an untried one, so even the greedy choice explores systematically. This optimism, not random ε moves, drives the agent through the dead end and around the loops early on; once the dead-end cells hold negative values they stop attracting it. Second, Q-learning is off-policy: it learns the values of the greedy policy no matter how the data was collected, so heavy exploration lowers training returns without changing what is learned. On this small deterministic map, ε therefore acts mainly as a cost setting, and the smallest value was best. I would expect ε to matter more with slip noise, a much larger map, or a decaying ε. Limitations: only three seeds per setting, one task, and one map.